

Sum of Maxima

Absolutely. This problem becomes **very simple once you understand one key idea: count how many times each element can become the maximum.**

1. What are we actually asked to calculate?

We have N distinct numbers.

We need to consider **every possible subset of exactly K elements.**

For each subset:

strength = maximum element in that subset

Then we need:

sum of maximum values of all K -sized subsets

For example:

```
a = [2, 3, 6, 8]
K = 3
```

All subsets of size 3:

```
{2,3,6} -> max = 6
{2,3,8} -> max = 8
{2,6,8} -> max = 8
{3,6,8} -> max = 8
```

So answer:

```
6 + 8 + 8 + 8 = 30
```

2. Why do we sort?

Suppose we sort:

```
[2, 3, 6, 8]
```

becomes:

```
2 3 6 8
0 1 2 3
```

Now look at 6.

If 6 is going to be the **maximum** of a subset, then every other element in that subset must be **smaller than 6.**

Because the array is sorted, all elements before 6 are smaller:

```
2 3 | 6 | 8
      ↑
    maximum
```

There are exactly 2 elements before 6.

Therefore, if $K = 3$, we need:

```
K - 1 = 2
```

other elements.

We can choose both from:

```
[2, 3]
```

Number of ways:

```
C(2, 2) = 1
```

So 6 contributes:

```
6 × 1 = 6
```

3. The most important observation

After sorting:

$$a[0] < a[1] < a[2] < \dots < a[N-1]$$

Consider $a[i]$.

If $a[i]$ is the maximum element of our K -sized subset:

- $a[i]$ **must be selected**
- We need another $K-1$ elements
- Those elements must come from the i elements before it

Those i elements are:

$$a[0], a[1], \dots, a[i-1]$$

So the number of ways is:

$$C(i, K-1)$$

Therefore:

$$\begin{aligned} \text{contribution of } a[i] \\ = a[i] \times C(i, K-1) \end{aligned}$$

And the final answer is:

$$\sum a[i] \times C(i, K-1)$$

for all i .

4. Let's understand with the sample

$$\begin{aligned} N &= 4 \\ K &= 3 \\ a &= [3, 6, 2, 8] \end{aligned}$$

Sort:

$$[2, 3, 6, 8]$$

Now calculate each element's contribution.

Element 2

There are 0 elements before it.

We need:

$$K - 1 = 2$$

elements.

Impossible:

$$C(0, 2) = 0$$

Contribution:

$$2 \times 0 = 0$$

Element 3

There is 1 element before it:

$$[2] \quad 3$$

Need 2 elements:

$$C(1, 2) = 0$$

Contribution:

$$3 \times 0 = 0$$

Element 6

There are 2 elements before it:

$$[2, 3] \quad 6$$

Need 2 elements:

$$C(2, 2) = 1$$

Contribution:

$$6 \times 1 = 6$$

Subset:

$$\{2, 3, 6\}$$

Element 8

There are 3 elements before it:

$$[2, 3, 6] \quad 8$$

Need 2 elements.

Choose any 2:

$$C(3, 2) = 3$$

The three subsets are:

$$\begin{aligned} &\{2, 3, 8\} \\ &\{2, 6, 8\} \\ &\{3, 6, 8\} \end{aligned}$$

All have maximum 8.

Contribution:

$$8 \times 3 = 24$$

Therefore:

$$\begin{aligned} \text{answer} &= 0 + 0 + 6 + 24 \\ &= 30 \end{aligned}$$

Exactly the sample answer.

5. But where does the total number of ways come from?

This is an important question.

The total number of K-sized subsets from N elements is:

$$C(N, K)$$

For example:

$$\begin{aligned} N &= 4 \\ K &= 3 \end{aligned}$$

Then:

$$C(4, 3) = 4$$

And indeed we had 4 subsets:

$$\begin{aligned} &\{2, 3, 6\} \\ &\{2, 3, 8\} \\ &\{2, 6, 8\} \\ &\{3, 6, 8\} \end{aligned}$$

But instead of generating all $c(N, K)$ subsets, we **group them according to their maximum**.

For our example:

Maximum	Number of subsets
2	0
3	0
6	$C(2, 2) = 1$
8	$C(3, 2) = 3$
Total	4

Notice:

$$1 + 3 = 4$$

which is exactly:

$$C(4, 3) = 4$$

This is why the method works.

Every K-sized subset has exactly one maximum, because all numbers are distinct.

So every subset gets counted **exactly once**.

6. Why $c(i, K-1)$ and not $c(i, K)$?

This is another very important point.

Suppose:

$$\begin{aligned} a &= [2, 3, 6, 8] \\ K &= 3 \end{aligned}$$

We're calculating the contribution of 6.

We want subsets like:

$$\{?, ?, 6\}$$

6 is already fixed.

So we only need to select:

$$K - 1 = 2$$

elements.

Available elements:

$$2, 3$$

Number of ways:

$$C(2, 2)$$

Therefore:

$$C(i, K-1)$$

7. Why must we choose only elements before $a[i]$?

Because $a[i]$ has to be the **maximum**.

Suppose we're considering 6:

$$\begin{array}{cccc} 2 & 3 & 6 & 8 \\ & & \uparrow & \end{array}$$

If we choose 8:

$$\{2, 6, 8\}$$

then 6 is **not** the maximum.

The maximum is 8.

Therefore k cannot be used when counting subsets whose maximum is k .

Sorting makes this incredibly easy:

Everything before $a[i]$ is smaller than $a[i]$.

So we simply choose $k-1$ elements from the first i elements.

8. Formula

After sorting:

```
a[0] < a[1] < ... < a[N-1]
```

For every i :

```
ways = C(i, K-1)
```

Contribution:

```
a[i] * C(i, K-1)
```

Therefore:

```
answer =
a[0] * C(0, K-1)
+
a[1] * C(1, K-1)
+
...
+
a[N-1] * C(N-1, K-1)
```

or simply:

```
answer = Σ a[i] * C(i, K-1)
```

9. How do we calculate $C(n, r)$?

We know:

```
C(n, r) = n! / (r! * (n-r)!)
```

But we're working modulo:

```
MOD = 1,000,000,007
```

We cannot directly divide under modulo.

So:

```
C(n, r)
= n! * inverse(r!) * inverse((n-r)!)
```

Using Fermat's Little Theorem:

```
inverse(x) = x^(MOD-2) mod MOD
```

Therefore:

```
C(n, r)
=
fact[n]
× invFact[r]
× invFact[n-r]
```

where:

```
fact[n] = n!
```

and:

```
invFact[n] = (n!)^(-1)
```

10. Why do we calculate factorials?

Since:

```
N <= 10000
```

we can precompute:

```
fact[0]
fact[1]
fact[2]
...
fact[N]
```

For example:

```
fact[0] = 1
fact[1] = 1
fact[2] = 2
fact[3] = 6
fact[4] = 24
...
```

Then:

```
C++
C(n,r) =
fact[n] * invFact[r] * invFact[n-r]
```

takes $O(1)$.

11. Complete C++ Code

```
C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll MOD = 1000000007;

// Fast exponentiation
ll power(ll a, ll b) {
    ll res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;

    while (T--) {
        int N, K;
        cin >> N >> K;

        vector<ll> a(N);

        for (auto &x : a)
            cin >> x;

        // 1. Sort the array
        sort(a.begin(), a.end());

        // 2. Precompute factorials
        vector<ll> fact(N + 1), invFact(N + 1);

        fact[0] = 1;

        for (int i = 1; i <= N; i++) {
            fact[i] = fact[i - 1] * i % MOD;
        }

        // inverse of N!
        invFact[N] = power(fact[N], MOD - 2);

        // Calculate inverse factorials
        for (int i = N - 1; i >= 0; i--) {
            invFact[i] = invFact[i + 1] * (i + 1) % MOD;
        }

        // Function to calculate nCr
        auto nCr = [&](int n, int r) -> ll {
```

```

        if (r < 0 || r > n)
            return 0;

        return fact[n]
            * invFact[r] % MOD
            * invFact[n - r] % MOD;
    };

    // 3. Calculate contribution of every element
    ll ans = 0;

    for (int i = 0; i < N; i++) {
        // a[i] is the maximum.
        // Choose K-1 elements from the i elements before it.
        ll ways = nCr(i, K - 1);

        ll contribution = (a[i] % MOD) * ways % MOD;

        ans = (ans + contribution) % MOD;
    }

    cout << ans << '\n';
}

return 0;
}

```

For the required format, change the final output to:

```

C++
cout << "Case #" << tc << ": " << ans << '\n';

```

and use:

```

C++
for (int tc = 1; tc <= T; tc++) {

```

12. Full code with Case

```

C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const ll MOD = 1000000007;

ll power(ll a, ll b) {
    ll res = 1;

    while (b) {
        if (b & 1)
            res = res * a % MOD;

        a = a * a % MOD;
        b >>= 1;
    }

    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;

    for (int tc = 1; tc <= T; tc++) {

        int N, K;
        cin >> N >> K;

        vector<ll> a(N);

        for (ll &x : a)
            cin >> x;

        // Sort so that elements before a[i]
        // are guaranteed to be smaller.
        sort(a.begin(), a.end());

        vector<ll> fact(N + 1);
        vector<ll> invFact(N + 1);

        // factorial
        fact[0] = 1;

        for (int i = 1; i <= N; i++) {
            fact[i] = fact[i - 1] * i % MOD;
        }

        // inverse factorial
        invFact[N] = power(fact[N], MOD - 2);

        for (int i = N - 1; i >= 0; i--) {
            invFact[i] = invFact[i + 1] * (i + 1) % MOD;
        }
    }
}

```

```

    }

    auto nCr = [&](int n, int r) -> ll {
        if (r < 0 || r > n)
            return 0;

        return fact[n]
            * invFact[r] % MOD
            * invFact[n - r] % MOD;
    };

    ll ans = 0;

    for (int i = 0; i < N; i++) {
        // a[i] is fixed as maximum.
        // Choose remaining K-1 elements
        // from the i smaller elements.
        ll ways = nCr(i, K - 1);

        ans = (ans + (a[i] % MOD) * ways) % MOD;
    }

    cout << "Case #" << tc << ": " << ans << '\n';
}

return 0;
}

```

13. Dry run of the entire algorithm

Take:

```

N = 5
K = 2
a = [10,20,30,40,50]

```

Already sorted.

Since $K = 2$, if an element is the maximum, we need:

```

K - 1 = 1

```

smaller element.

10

```

i = 0
C(0,1) = 0

```

Contribution:

```

10 × 0 = 0

```

20

```

i = 1
C(1,1) = 1

```

Contribution:

```

20 × 1 = 20

```

Subset:

```

{10,20}

```

30

```

C(2,1) = 2

```

Contribution:

```

30 × 2 = 60

```

Subsets:

```

{10,30}
{20,30}

```


40

$$C(3,1) = 3$$

Contribution:

$$40 \times 3 = 120$$

50

$$C(4,1) = 4$$

Contribution:

$$50 \times 4 = 200$$

Total:

$$0 + 20 + 60 + 120 + 200 = 400$$

Sample answer:

$$400$$

14. The intuition you should remember

Don't think:

"I need to generate every K-sized subset."

That would be enormous.

Instead think:

"For each number, how many subsets have this number as their maximum?"

After sorting:

```

smaller elements | current element
                  ↑
                maximum
    
```

For $a[i]$:

```

      a[i]
      ↑
     MAX
[ i smaller elements ]
    
```

$a[i]$ is already selected.

We need $K-1$ more.

Therefore:

$$\begin{aligned} &\text{number of subsets where } a[i] \text{ is maximum} \\ &= C(i, K-1) \end{aligned}$$

Then:

$$\text{answer} += a[i] * C(i, K-1)$$

That's the entire core idea.

15. Complexity

Sorting:

$$O(N \log N)$$

Factorials:

$$O(N)$$

Each nCr :

$O(1)$

Loop over elements:

$O(N)$

So total:

$O(N \log N)$

per test case.

Memory:

$O(N)$

The key formula to memorize for this problem is:

After sorting, $a[i]$ is the maximum in exactly $C(i, K-1)$ K -sized subsets.

So:

$ANSWER = \sum a[i] \times C(i, K-1)$

This is a very common "**sort + contribution counting + combinations**" pattern in combinatorics problems.

Yes — this is the **most important point** to understand. i itself is **NOT the number of ways**.

i is the **number of elements available to choose from**.

Let's break it down carefully.

Suppose sorted array is

$[2, 3, 6, 8]$
 $0 \ 1 \ 2 \ 3$

Consider $a[i] = 6$.

Here:

$i = 2$

Why?

Because there are exactly **2 elements before 6**:

$[2, 3] \ 6$
 $\uparrow \ \uparrow \ \uparrow$
 2 elements

Now, if 6 must be the maximum of a hand of size $K = 3$:

$\{ ?, ?, 6 \}$

6 is already fixed.

So we need:

$K - 1 = 2$

more elements.

Those 2 elements must be chosen from:

$[2, 3]$

There are 2 available elements, and we need to choose 2.

Therefore:

$C(2, 2) = 1$

Notice the meaning:

$C(i, K-1)$
 $\uparrow \ \uparrow$
 $| \ \ |$

```
| how many elements we need
|
| how many elements are available
```

So i is not "number of ways".

It is:

the number of smaller elements available before $a[i]$.

Another example makes this clearer

Consider:

```
[2, 3, 6, 8]
```

Now consider **8**.

```
i = 3
```

because:

```
[2, 3, 6] 8
  ↑  ↑  ↑
  3 elements
```

Suppose $K = 3$.

We need:

```
K - 1 = 2
```

elements along with **8**.

We have 3 choices available:

```
2, 3, 6
```

So:

```
C(3,2) = 3
```

The three hands are:

```
{2,3,8}
{2,6,8}
{3,6,8}
```

Therefore **8** appears as the maximum in **3 different hands**.

So remember this distinction

For $a[i]$:

```
i = number of elements smaller than a[i]
```

Then:

```
K - 1 = number of additional elements required
```

Therefore:

```
C(i, K-1)
```

means:

"From the i smaller elements, how many ways can I select the remaining $K-1$ cards?"

And **that result** is the number of ways.

Visual

```
Sorted:
a[0] a[1] a[2] ... a[i]
  <  <  <  ...  ↑
```

```

                                maximum
<---- i elements ---->
Choose K-1 from these i elements
    ↓
    C(i, K-1)
    ↓
number of hands

```

So the formula:

```

C++
a[i] * C(i, K-1)

```

means:

```

strength of this maximum
      x
number of hands having this maximum

```

That's why we multiply them.

ChatGPT can make mistakes. Check important info.